- The union of two graphs `G=(V,E)` and `G'=(V',E')`: we just compute the union of both vertices and edges set, to obtain `G"=(V U V', E U E')`.

This representation prevents the inconsistencies we talked about, guarantees that we won't get malformed graphs, but also allows to formally define algorithms as transformations on graphs and mathematically prove their correctness.

Trying to understand graphs as an algebraic type is an useful exercise to grasp a deeper understanding of this data structure; at the same time, we need to acknowledge that, considering the overhead for these operations, practical uses are limited, and mostly relegated to functional languages providing pattern matching on types, like Scala, Haskell, or Clojure[2].

14.1.3 Pseudo code

To complete the discussion, listing 14.1 provides an overview of the class that we'll use, in the book, for graphs. It uses adjacency lists and models edges and vertices as classes, allowing to implement different types of graphs by changing the details of these models (for instance, allowing weighted edges).

Listing 14.1 The Graph class

```
class Vertex
  #type string
  label

class Edge
```